U.PORTO
FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

Academia de Física 2026

Experiência 2 – Expansão térmica [Guião de Resolução]

Neste documento encontrarás uma descrição do procedimento que deverás considerar durante a análise numérica dos dados obtidos experimentalmente para a determinação do coeficiente de expansão térmica (α_L) de uma amostra de Ouro.

No Jupyter Notebook `Expansão térmica – Notebook alunos - 2026` apenas necessitarás de modificar as células de código destacadas a vermelho.

(a) Extração dos dados

In [ ]:
```
# Temperatura da medição
T = 300 # K

# nome do ficheiro de dados
filename = directory + foldername + r'\Bulk'+str(T)+r'K.chi'

# load dos dados a partir do ficheiro
data = np.genfromtxt( filename , delimiter = ' ', skip_header = 1)

teta2_300 = data[:,0] # ângulo
I_300 = data[:,1] # Intensidade
```

(b) Identificação do máximo de intensidade

In [ ]:
```
# Identificação do ângulo e do máximo de intensidade
I_300_max = # máximo de intensidade
index_max = # índice associado ao máximo de intensidade
teta2_300_max = teta2_300[ index_max ] # ângulo associado ao máximo de intensidade

# Na representação de difractogramas é comum a sua normalização ao valor do máximo de intensidade
I_300_norm = I_300 / I_300_max
```

Figura 1: (a) Exemplo de célula a **não editar**. **(b)** Exemplo de célula a **editar.**

Descrição geral

Para determinar o coeficiente de expansão térmica do Ouro é necessário obter a dependência na temperatura do parâmetro de rede da sua estrutura cristalina – $a(T)$. Para o efeito, será necessário analisar individualmente os padrões de difração obtidos para cada uma das temperaturas em estudo ($T \in [100, 300]\ K$). Como podes imaginar, <u>se não for devidamente automatizado</u>, este será um processo muito duradouro.

Posto isto, na **primeira parte** desta atividade tentaremos obter o parâmetro de rede do Ouro para uma temperatura específica ($T = 300\ K$, p.e.), tendo em conta que depois, na **segunda parte**, pretendemos automatizar o processo de análise para as restantes temperaturas.

No **final**, é proposto um **desafio**, onde será aprofundada a análise de $a(T)$. Conforme verás, a amostra estudada não apresenta uma expansão térmica perfeitamente linear, possuindo duas regiões com comportamento linear distinto. Do ponto de vista tecnológico, torna-se essencial a análise individual dos dois regimes.

Nota: Se tiveres dúvidas sobre como utilizar alguma das funções do *numpy* ou do *scipy* mencionadas adiante pesquisa pelo seu nome no Google, ou então chama um dos monitores.

Determinação do parâmetro de rede do Ouro a $300\ K$

Parte 1 – Identificação do pico de máxima intensidade

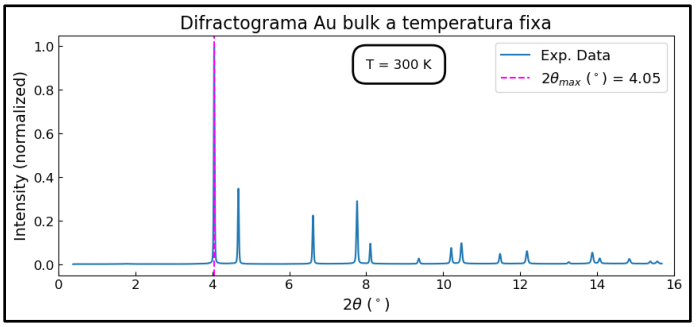
- Extração dos dados
  - Utilizando a função do *numpy* `np.genfromtxt()` extraímos os valores do ficheiro de dados `Bulk300K.chi` para um *array* com o nome `data` com dimensões $N \times 2$ ($N \equiv$ nº de linhas, $2 \equiv$ nº de colunas).
    - Primeira coluna $\equiv$ valores de 2θ
    - Segunda coluna $\equiv$ valores de intensidade (I)
  - Com as linhas de código abaixo conseguimos então guardar os valores de 2θ a $300\ K$ na variável `teta2_300` e os valores de I a $300\ K$ na variável `I_300`.

```
teta2_300 = data[:,0] # ângulo
I_300 = data[:,1] # Intensidade
```

- **Identificação do máximo de intensidade**
  - Utiliza a função do *numpy* `np.max()` para determinar o valor do máximo de intensidade (`I_300_max`)
  - Determina o índice associado ao máximo de intensidade (`index_max`) utilizando a função `np.argmax()`
  - Assim conseguirás determinar o ângulo associado ao máximo de intensidade $(2\theta_{max})$ que fica associado à variável `teta2_300_max`, recorrendo à linha de código:

```
teta2_300_max = teta2_300[ index_max ]
```

- Visualização geral dos dados e do pico de máxima intensidade
  - Resultado esperado:



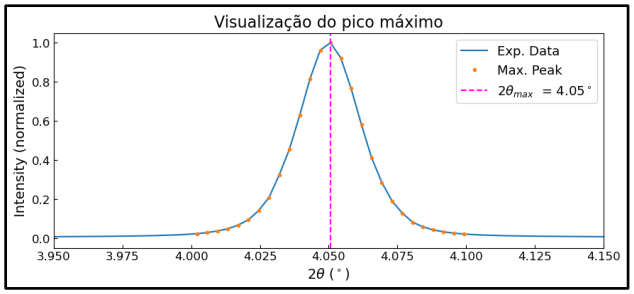
- **Segmentação dos dados a uma região centrada em $2\theta_{max}$**
  - Começa por determinar os índices do array `teta2_300` onde se verifica a condição $2\theta_{max} - \frac{\delta\theta}{2} \leq 2\theta \leq 2\theta_{max} + \frac{\delta\theta}{2}$
    - Utiliza a função do *numpy* `np.where()` para obter um array (`index_peak`) onde ficam guardados os índices pretendidos

```
index_peak = # np.where( condição a verificar )[0]
```

  - Restringe os arrays `teta2_300` e `I_300_norm` aos valores associados aos índices obtidos, ou seja, ao pico de máxima intensidade
    - `teta2_300_peak` ≡ array com os valores de 2θ associados ao pico de máxima intensidade
    - `I_300_norm_peak` ≡ array com os valores de I (normalizados) associados ao pico de máxima intensidade

```
# Arrays de teta2 e I associados a esses ângulos
teta2_300_peak = teta2_300[index_peak]
I_300_norm_peak = # preencher
```

- Visualização do pico de máxima intensidade
  - Resultado esperado:



Parte 2 – Ajuste Gaussiano ao pico de máxima intensidade

- **Função de ajuste – Gaussiana**
  - Define a função `Gaussian()` que, introduzindo os valores de 2θ do pico a ajustar, devolve os valores de intensidade que seguem o perfil de uma Gaussiana $[G(2\theta)]$ com parâmetros I_{peak} , $2\theta_{peak}$, ω_{peak}
    - Apenas necessitas de associar `G` à expressão apresentada no enunciado

- Função que efetua o ajuste considerando o método dos mínimos quadrados
  - Definição da função `curve_fitting()` que efetua o ajuste aos dados experimentais

- **Ajuste da curva**
  - Começa por determinar os valores dos parâmetros de ajuste, utilizando a função `curve_fitting()`
    - `parms_fit` – lista com os valores dos parâmetros de ajuste
    - `u_parms_fit` – lista com os valores das incertezas dos parâmetros de ajuste
  - Guarda os valores dos parâmetros de ajuste e suas incertezas em variáveis
    - Este procedimento é feito pelas seguintes linhas de código

```
I_0_300 , teta2_0_300 , w_300 = parms_fit
u_I_0_300 , u_teta2_0_300 , u_w_300 = u_parms_fit
```

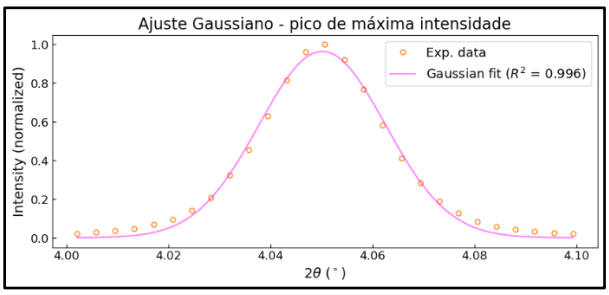
    - `I_0_300` ≡ variável com o valor de I_{peak} a 300 K
    - `teta2_0_300` ≡ variável com o valor de $2\theta_{peak}$ a 300 K
    - `w_300` ≡ variável com o valor de ω_{peak} a 300 K
  - Avalia a qualidade do ajuste, calculando o coeficiente de Pearson (R^2). Essencialmente, quanto mais próximo de 1 for o valor de R^2 menor será o desvio dos valores de ajuste relativamente aos valores obtidos experimentalmente.
    - As células seguintes calculam este valor:

```
# Avaliação da qualidade do ajuste - Cálculo do coeficiente de Pearson
r_300 , u_r_300 = sps.pearsonr( Gaussian( teta2_300_peak , I_0_300 , teta2_0_300 , w_300 ) , I_300_norm_peak )
R_300 = r_300 ** 2
# Quanto mais próximo de 1 for o valor do coefciente de Pearson, melhor será a qualidade do ajuste
```

  - Visualiza os valores dos parâmetros de ajuste.
    - Imprime os parâmetros de ajuste I_{peak} , $2\theta_{peak}$, ω_{peak} guardados nas variáveis `I_0_300` , `teta2_0_300` , `w_300`, respetivamente, assim como as suas incertezas.
    - Para isso, considera uma estrutura semelhante à seguinte linha de código:

```
# Print dos parâmetros de ajuste
print('I_peak = {:.2f} +- {:.2f}'.format(I_0_300 , u_I_0_300))
```

- Visualização do ajuste
  - Resultado esperado:



Parte 3 – Estimativa do parâmetro de rede (a) e do tamanho de cristalite (D)

- **Estimativa do parâmetro de rede**
  - Considerando a expressão apresentada no Notebook para o parâmetro de rede (a) determina o seu valor a partir dos parâmetros de ajuste.
    - Para calculares $\sin\theta_{111}$ utiliza a função `np.sin()` do *numpy*
    - `a_300` $\equiv$ variável onde fica guardado o valor de a para $T = 300\ K$
    - Output esperado:

```
Valor estimado para o parâmetro de rede
 a = 4.081 A
```

- **Estimativa do tamanho de cristalite**
  - Considerando a expressão apresentada no Notebook para o tamanho de cristalite (D) calcula o seu valor a partir dos parâmetros de ajuste.
    - Para calculares $\cos\theta_{111}$ utiliza a função `np.cos()` do *numpy*
    - `D_300` $\equiv$ variável onde fica guardado o valor de D para $T = 300\ K$
    - Output esperado:

```
Valor estimado para o tamanho de cristalite
 D = 5.373 A
```

Determinação do coeficiente de expansão térmica

Nesta parte o objetivo passa por automatizar o processo de análise para as restantes temperaturas, seguindo exatamente os mesmos passos considerados para o padrão de difração a $300\ K$. Para o efeito, vamos tirar proveito da capacidade do Jupyter Notebook de armazenar na sua memória local dados (*array* ou um simples valor) associados a uma determinada variável (com um nome específico). Nesse sentido usaremos a seguinte estrutura:

- Quando quisermos guardar na memória um certo tipo de dados:

```python
# nomes das variáveis nas quais vamos guardar os arrays teta2 e I
teta2_name = "teta2_" + str(int(T))
I_name = "I_" + str(int(T))

# Alocação na memória dos arrays de dados
myVars = locals()
# myVars.__setitem__( nome da variável , array a associar à variável )
myVars.__setitem__(teta2_name , teta2)
myVars.__setitem__(I_name , I)
```

- Quando quisermos utilizar o valor guardado anteriormente, atribuímo-lo a uma variável volátil (volátil porque muda de valor a cada iteração dos ciclos **for**):

```python
# Atribuição dos arrays previamente guardados a variáveis voláteis
# variavel volátil = globals()[ nome da variável na memória com os dados pretendidos ]

teta2_name = "teta2_" + str(int(T))
I_name = "I_" + str(int(T))

teta2 = globals()[teta2_name]
I = globals()[I_name]
```

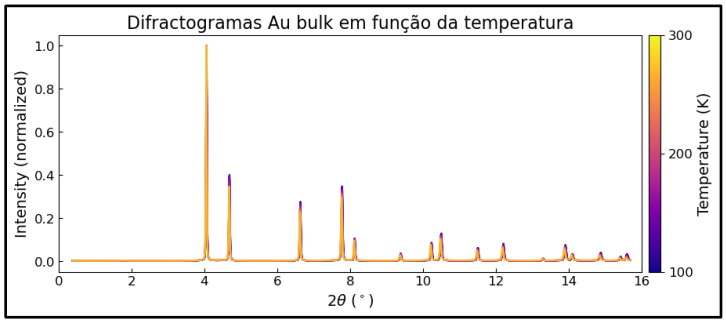
Parte 4 – Obter dependência de α e de D com a temperatura

<u>Identificação do pico de máxima intensidade</u>

- Extração dos dados
  - O procedimento abaixo é efetuado para cada iteração do ciclo **for**, ou seja, para cada índice "*n*" e temperatura "*T* " do array "*Ts*" (array com os valores de temperatura considerados)
  - Utilizando a função do *numpy* `np.genfromtxt()` extraímos os valores do ficheiro de dados `BulkTK.chi` para um *array* com o nome `data`
  - Guardamos os valores de 2θ e de I nas variáveis voláteis `teta2` e `I`, respetivamente.
  - Definimos o nome das variáveis que vamos guardar na memória associadas aos valores dos arrays `teta2` e `I`
    - `teta2_name` – *string* com o nome da variável que guarda os valores de 2θ à temperatura T
    - `I_name` – *string* com o nome da variável que guarda os valores de I à temperatura T
  - Guardamos essas variáveis na memória

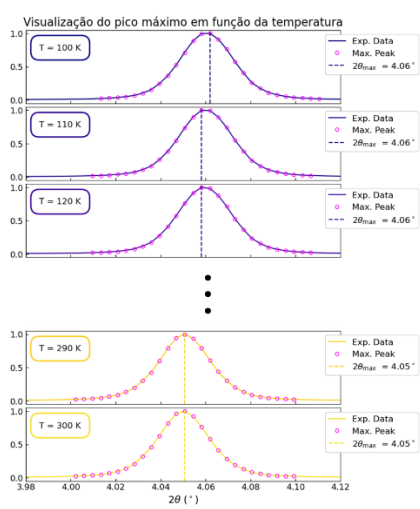
- **Identificação do máximo de intensidade**
  - Para cada valor de temperatura T, atribuímos o correspondentes valores de 2θ e de I guardados na memória às variáveis voláteis `teta2` e `I`, respetivamente.
  - Utilizando a função do *numpy* `np.max()` determinamos o valor do máximo de intensidade (`I_max`) ***[a fazer]***
  - Determinamos o índice associado ao máximo de intensidade (`index_max`) utilizando a função `np.argmax()` ***[a fazer]***
  - Determinamos o ângulo associado ao máximo de intensidade ($2\theta_{max}$) que fica associado à <u>variável volátil</u> `teta2_max` ***[a fazer]***
  - Definimos os nomes das variáveis que queremos guardar na memória
    - `I_norm_name` – *string* com o nome da variável que guarda os valores normalizados de intensidade (*array* `I_norm`)
    - `teta2_max_name` – *string* com o nome da variável que guarda o valor do ângulo $2\theta_{max}$ associado a `teta2_max`

- Visualização geral dos dados
  - Resultado esperado:



- **Segmentação dos dados a uma região centrada em $2\theta_{max}$**
  - Para cada valor de temperatura T, começa por atribuir os valores guardados na memória (2θ, I_{norm} e $2\theta_{max}$) a variáveis voláteis (*arrays* `teta2`, `I_norm`, `teta2_max`), respeitando nome que deste anteriormente a essas variáveis (`teta2_name`, `I_norm_name`, `teta2_max_name`)
  - Determina os índices do array `teta2` onde se verifica a condição $2\theta_{max} - \frac{\delta\theta}{2} \leq 2\theta \leq 2\theta_{max} + \frac{\delta\theta}{2}$
    - Conforme anteriormente, utiliza a função do *numpy* `np.where()` para obter um array (`index_peak`) onde ficam guardados os índices pretendidos
  - Atribui às variáveis `teta2_peak` e `I_norm_peak`, os arrays `teta2` e `I_norm` restringidos aos índices obtidos, ou seja, ao pico de máxima intensidade
  - Define os nomes (associados às *strings* `I_norm_peak_name` e `teta2_peak_name`) das variáveis (*arrays* `I_norm_peak` e `teta2_peak`) que queremos guardar na memória

- Visualização dos picos de máxima intensidade
  - Resultado esperado:



Ajuste Gaussiano ao pico de máxima intensidade

- Criação de arrays para guardar os parâmetros de ajuste em função da temperatura
  - Utilizar a função `np.zeros()` do *numpy* para criar arrays com a dimensão do *array* `Ts` (array com o valor de temperaturas) onde cada elemento corresponderá aos parâmetros de ajuste da curva Gaussiana ao pico de máxima intensidade
    - `I_0_T` – *array* com os valores de I_{peak} em função da temperatura
    - `teta2_0_T` – *array* com os valores de $2\theta_{peak}$ em função da temperatura
    - `w_T` – *array* com os valores de ω_{peak} em função da temperatura
    - `R_T` – *array* com os valores de R^2 (coeficiente de Pearson) em função da temperatura

- **Ajuste das curvas**
  - Para cada valor de temperatura T, começa por atribuir os valores do **primeiro pico** guardados na memória (2θ e I_{norm}) a variáveis voláteis (*arrays* `teta2_peak` e `I_norm_peak`), respeitando nome que deste anteriormente a essas variáveis (guardado nas *strings* `teta2_peak_name` e `I_norm_peak_name`)
  - Determina os valores dos parâmetros de ajuste, utilizando a função `curve_fitting(x , y , func)`
    - `x` ≡ ordenada do ajuste
    - `y` ≡ abcissa do ajuste
    - `func` ≡ nome da função que queremos ajustar aos dados
  - Guarda os valores dos parâmetros de ajuste e suas incertezas em variáveis voláteis
    - Este processo é feito pelas linhas de código:

```
# Grandezas obtidas a partir dos parâmetros de ajuste
I_0 , teta2_0 , w = parms_fit
u_I_0 , u_teta2_0 , u_w = u_parms_fit
```

  - Atribui cada um dos parâmetros obtidos no ajuste ao respetivo elemento do array correspondente.
    - Por exemplo, para atribuir o valor de I_{peak} (guardado na variável `I_0`) à temperatura T deverás fazer:

```
I_0_T[n] = I_0 * 1.
```

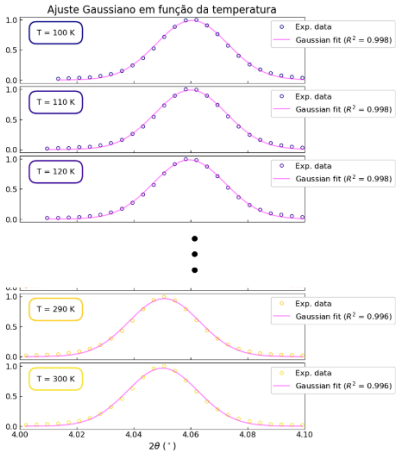
  - Avalia a qualidade do ajuste, calculando o coeficiente de Pearson (R^2).
    - Recorda o que fizeste <u>no caso da temperatura $T = 300\ K$</u>:

```
# Avaliação da qualidade do ajuste - Cálculo do coeficiente de Pearson
r_300 , u_r_300 = sps.pearsonr( Gaussian( teta2_300_peak , I_0_300 , teta2_0_300 , w_300 ) , I_300_norm_peak )
R_300 = r_300 ** 2
# Quanto mais próximo de 1 for o valor do coefciente de Pearson, melhor será a qualidade do ajuste
```

  - Imprime os valores dos parâmetros de ajuste e avalia a sua coerência
    - Por exemplo, para $T = 100\ K$ deverás obter:

```
        Temperatura = 100 K
I_peak = 0.99 +- 0.01
teta2_peak (º) = 4.060 +- 0.000
w_peak = 0.015 +- 0.000
R^2 = 0.9977
---------------------------------------------
```

- Visualização dos ajustes
  - Resultado esperado:

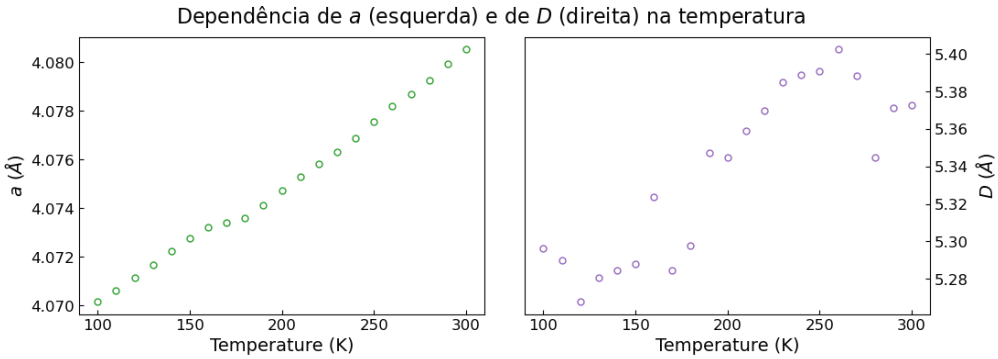


<u>Estimativa do parâmetro de rede e do tamanho de cristalite</u>

- **Estimativa do parâmetro de rede**
  - Calcula a dependência do parâmetro de rede na temperatura considerando a fórmula determinada anteriormente:
    - $a(T) = \frac{\sqrt{3}}{2} \cdot \frac{\lambda}{\sin \theta_{peak}(T)}$
    - `a_T` ≡ variável (*array*) onde ficam guardados os valores de $a(T)$
    - Ao usar a função `np.sin()` não te esqueças de colocar o argumento da função em radianos

- **Estimativa do tamanho de cristalite**
  - Calcula a dependência do parâmetro de rede na temperatura considerando a fórmula determinada anteriormente:
    - $D(T) = \frac{K \cdot \lambda}{\beta_{peak} \cdot \cos \theta_{peak}(T)}$
    - `D_T` ≡ variável (*array*) onde ficam guardados os valores de $D(T)$
    - Ao usar a função `np.cos()` não te esqueças de colocar o argumento da função em radianos

- **Visualização da sua dependência na temperatura**
  - Preenche o que está em falta na visualização gráfica.
  - Resultado esperado:



Parte 5 – Determinação do coeficiente de expansão térmica

- **Definição da função de ajuste linear**
- **Definição da função de ajuste linear**
  - Define a função `linear()` que, introduzindo os valores de T considerados, devolve os valores de $\frac{\Delta a}{a}$ que seguem uma reta com declive m e ordenada na origem b.
    - Apenas necessitas de associar `y` à expressão típica de uma reta

- **Ajuste linear à dependência do parâmetro de rede na temperatura**
  - Começa por definir $\frac{\Delta a}{a}(T)$ considerando a expressão apresentada no enunciado e recorrendo à variável `delta_a_a_T`
  - Determina os valores dos parâmetros de ajuste, utilizando a função `curve_fitting()`
    - `parms_fit` – lista com os valores dos parâmetros de ajuste
    - `u_parms_fit` – lista com os valores das incertezas dos parâmetros de ajuste
  - Guarda os valores dos parâmetros de ajuste e suas incertezas em variáveis
    - Este procedimento é feito pelas seguintes linhas de código

```
alpha , b = parms_fit
u_alpha , u_b = u_parms_fit
```

    - `alpha` ≡ variável com o valor de α_T
    - `b` ≡ variável com o valor da ordenada na origem
    - `w_300` ≡ variável com o valor de ω_{peak} a 300 K
  - Avalia a qualidade do ajuste, calculando o coeficiente de Pearson (R^2). Essencialmente, quanto mais próximo de 1 for o valor de R^2 menor será o desvio dos valores de ajuste relativamente aos valores obtidos experimentalmente.
    - As células seguintes calculam este valor:

```
# Avaliação da qualidade do ajuste - Cálculo do coeficiente de Pearson
r , u_r = sps.pearsonr( linear( Ts , alpha , b ) , delta_a_a_T )
R_alpha = r ** 2
```

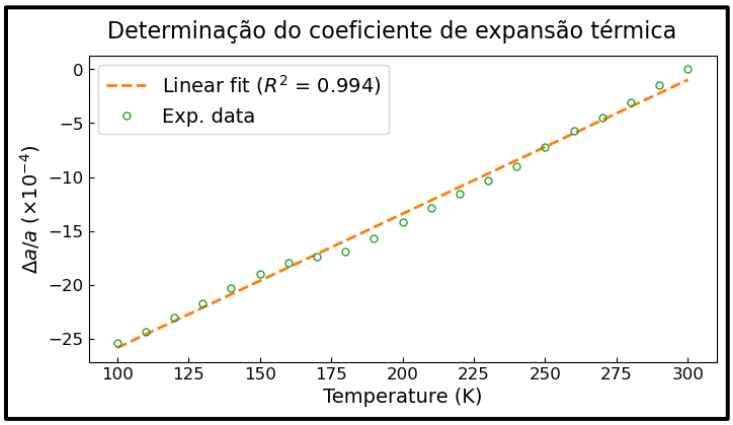
- Visualiza os valores dos parâmetros de ajuste.
  - Imprime os parâmetros de ajuste α_T , b , R^2 guardados nas variáveis `alpha` , `b` , `R_alpha` , respetivamente, assim como as suas incertezas.
  - Para isso, preenche devidamente as linhas de código:

```
# Print dos parâmetros de ajuste - PREENCHER
print('alpha = {:.2f} +- {:.2f} E-5 K^-1'.format( * 1e5 ,  * 1e5))
print('b = {:.3f} +- {:.3f} E-3'.format(  * 1e3 ,  * 1e3 ))
print('R^2 = {:.4f}'.format(  ))
print('----------------------------------------------')
```

  - Resultado esperado:

```
alpha = 1.24 +- 0.02 E-5 K^-1
b = -3.827 +- 0.048 E-3
R^2 = 0.9935
----------------------------------------------
```

- Visualização do ajuste
  - Resultado esperado:



Desafio final – Estudo dos diferentes regimes de expansão térmica

Neste desafio, testa a tua independência e pensamento crítico, analisando de forma mais detalhada a evolução do parâmetro de rede com a temperatura. A amostra estudada apresenta duas regiões com diferente comportameto linear, e, portanto, com valores distintos de α_T. Ao introduzir este material num contexto tecnológico torna-se crucial estabelecer qual será a dilatação térmica experienciada em cada regime de temperatura.

Utilizando as ferramentas aprendidas até esta parte, durante a resolução do desafio deverás tentar responder às seguintes questões:

- De que forma poderia automatizar a segmentação para amostras com diferentes regimes lineares, em gamas de temperatura arbitrárias?
- Como deverá ser definido $\frac{\Delta a}{a}(T)$ em cada gama de temperaturas?

- O que estará na origem da diferença no comportamento térmico do material?

Para referência, de α_T deverão manter-se na mesma ordem de grandeza dos determinados anteriormente na parte 5.

Reitera-se que este desafio visa testar as tuas aprendizagens no final das sessões computacionais. No entanto, sempre que tiveres dúvidas podes e deves chamar os teus monitores. Boa sorte!